

Module 06 Lab - Regression and Classification

Models Objective: To understand the difference between regression and classification and to build your first linear models for both tasks. **In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. Classification This is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not. **In this lab, we will tackle one of each.**

Part 2: Linear Regression **Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the **Fare** of a Titanic passenger based on their **Age** and **Pclass**.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets')
# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)
# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'
X_reg = df[features]
y_reg = df[target_reg]
# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
X_reg, y_reg, test_size=0.2, random_state=42
)

```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1.

Create an instance of the `LinearRegression` model.2. Train the

model using the training data (`X_train_reg`, `y_train_reg`).3.

Make predictions on the test data.4. Evaluate the model using

`mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```

# --- ENTER YOUR CODE HERE ---
# 1. Create the model instance
import numpy as np
# lr_model = ...
lr_model = LinearRegression()
# 2. Train the model# lr_model.fit(...)
lr_model.fit(X_train_reg, y_train_reg)
# 3. Make predictions# y_pred_reg = lr_model.predict(...)
y_pred_reg = lr_model.predict(X_test_reg)
# 4. Evaluate the model# mse = mean_squared_error(y_test_reg, y_pred_reg)
# print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
# print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is

```

Mean Squared Error for Fare Prediction: 3364.92

The square root of this is 58.01, meaning our model is off by about \$48 on av

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.*

Problem: We will predict whether a passenger **Survived** based on their **Age**, **Pclass**, and **Sex**.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

# --- Data Preparation ---
# Encode the 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'
X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data into training and testing sets
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42
)

# --- Model Pipeline ---

# 1. Create the model instance
log_reg_model = LogisticRegression()

# 2. Train the model
log_reg_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
y_pred_cls = log_reg_model.predict(X_test_cls)

# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Classification: {accuracy:.2f}")

# Optional: Detailed classification report
```

```
print("\nClassification Report:")
print(classification_report(y_test_cls, y_pred_cls))
```

Accuracy for Survival Classification: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.78	0.79	87
1	0.67	0.70	0.68	56
accuracy			0.75	143
macro avg	0.74	0.74	0.74	143
weighted avg	0.75	0.75	0.75	143

Task 2: Train and Evaluate a Logistic Regression Model**Your

Task:**1. Create an instance of the `LogisticRegression` model.2.

- ✓ Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
# --- ENTER YOUR CODE HERE ---
# 1. Create the model instance
log_model = LogisticRegression()

# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

Accuracy for Survival Prediction: 74.83%



Instructions: Answer the following questions in this markdown cell.

1. In your own words, what is the key difference between a regression problem and a classification problem?
2. The `LinearRegression` model has an attribute called `.coef_`. After you

train the model, print `lr_model.coef_`. What do these numbers represent?

3. Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?

[ENTER YOUR ANSWERS HERE]

1. Classification has to do with yes or no aka discrete categories or classes while regression deals with continuous numerical values.
2. They represent the relationship between each feature and the target variable.
3. Measuring MSE between predicted numbers and actual numbers is ideal for continuous values and `accuracy_score` measures the percentage of correct discrete choices made out of these predictions. Fare price is a continuous dollar amount so predicting the exact price down to the penny will not be accurate. For regression, we look at how close our estimate is to the real value.